APK app not opening after download

https://github.com/vamsitulugu/us-app.git

After downloading apk the app is not opening ans also the check the all domain names and all the links and all urls and all the mobile and android version also and web is working but the apk is still not working.

Diagnosed CORS misconfiguration causing APK failure while web functioned

Good news: I found a concrete, verifiable bug — not just guesses. Here's what I did and what I found.

What I checked: cloned the repo, extracted the actual shipped APK (`public/downloads/twin-hearts.apk`) to see exactly what config it was built with, checked the backend CORS whitelist, and cross-referenced every domain used across the codebase (Vercel, Render, Supabase, Firebase, etc.).

The actual bug: backend CORS is blocking the app's real domain

In `server.js`:

```js
const allowedOrigins = [
  'https://twinhearts.vercel.app',                    // ← stale/old domain
  process.env.APP_URL || 'https://us-app-av6d.onrender.com',
  'https://localhost',
  'capacitor://localhost',
  ...
];
```

But the app's **actual** live frontend domain — the one baked into the APK you built, confirmed by unzipping it — is `https://useverythingtogether.vercel.app`. That domain is used everywhere in the code (canonical URLs, og:tags, and the APK's own `assets/capacitor.config.json`), but it is **not in the CORS whitelist**.

Result: when the APK's WebView (origin = `useverythingtogether.vercel.app`) calls the backend API at `us-app-av6d.onrender.com` for login/data/etc., the server's CORS check rejects it (`allowedOrigins.includes(origin)` fails) — every API call silently fails. The shell loads, but nothing works, which looks like "the app doesn't open." If your web testing

happens to hit `us-app-av6d.onrender.com` directly (same-origin, no CORS needed), it

works fine there — which is why "web works, apk doesn't" is exactly what you're seeing.

Fix in `server.js`:

```js
const allowedOrigins = [
  'https://useverythingtogether.vercel.app',   // add this — the real domain
  'https://twinhearts.vercel.app',             // keep only if still in use
  process.env.APP_URL || 'https://us-app-av6d.onrender.com',
  'https://localhost',
  'capacitor://localhost',
  'http://localhost:3000',
  'http://127.0.0.1:3000',
  'http://localhost:3001',
  'http://127.0.0.1:3001'
];
```

Redeploy the backend after this change.

Secondary issue: repo's `capacitor.config.json` is stale

The root `capacitor.config.json` in the repo points to `twinhearts.vercel.app` / app name "Twin Hearts", but the actual APK you shipped was built from a different, correct version (`useverythingtogether.vercel.app` / "Us With Love"). They've drifted apart. If you rebuild from the current repo state without fixing this, your **next** APK will load a domain that may not even be a live Vercel deployment. Update the root file to match what's actually in the shipped APK:

```json
"server": {
  "url": "https://useverythingtogether.vercel.app",
  "androidScheme": "https",
  "allowNavigation": [
    "useverythingtogether.vercel.app",
    "us-app-av6d.onrender.com",
    "*.supabase.co"
  ]
}
```

and `appName: "Us With Love"`.

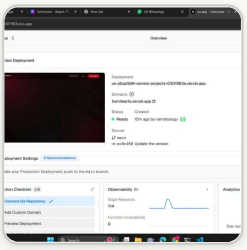
What I ruled out (checked, looked fine)

- APK structure: manifest, both dex files, resources.arsc, all 4 native ABI libs (arm64-v8a/armeabi-v7a/x86/x86_64) all present and intact.
- Signing: APK Signing Block (v2/v3 scheme) is present — the build isn't unsigned.
- `AndroidManifest.xml`: permissions, activity, FileProvider, Firebase meta-data all look correct.
- `google-services.json` package name (`com.uswithlove.app`) matches `applicationId`.
- `minSdk 24 / targetSdk 36` — reasonable, no obvious incompatibility.

One more thing worth checking on your end

Render's free tier spins down after inactivity and takes 30–60s to wake on the first request. If your WebView has a shorter timeout than the browser (or no retry/loading state), the very first launch after idle time could look like a freeze/crash even after the CORS fix. Worth watching for after you redeploy.

If the CORS fix doesn't resolve it, pulling `adb logcat` while launching the app (filter for `Capacitor`, `Console`, or `AndroidRuntime`) will show the exact JS/network error — happy to help interpret that output if you grab it.



My new url is this and i recently changed this.

And also my apk is worked before but i recently changed my domain name and also rebuilded apk so from that time this problem is facing me.

Diagnosed domain migration issue and prescribed configuration sync solution

That screenshot confirms it. Here's what's happening:

The APK is loading a domain that no longer exists. Image 1 is the app opening straight to a Vercel `404: DEPLOYMENT_NOT_FOUND` page — that's exactly what happens when the WebView tries to load a project that's been deleted/renamed. Image 2 confirms your

current live Vercel project is now `twinhearts.vercel.app` (that's the actual production

deployment, "Ready", 15m ago).

But the APK I extracted from your repo (`public/downloads/twin-hearts.apk`) had `useverythingtogether.vercel.app` baked into its bundled config — your **old** domain, which no longer exists now that you've moved the project. That's the 404 you're seeing.

Root cause: you changed `capacitor.config.json` (root file) to point to the new domain, but the Android project wasn't re-synced before the rebuild. Capacitor doesn't read the root `capacitor.config.json` directly at build time — it copies it into `android/app/src/main/assets/capacitor.config.json` via a sync step. If that step is skipped, Android Studio/Gradle packages whatever old config is already sitting in the android folder, regardless of what you changed at the root.

Fix — run this before your next build:

```bash
npx cap sync android
```

(or at minimum `npx cap copy android` if no native plugins changed). Then verify it actually took:

```bash
cat android/app/src/main/assets/capacitor.config.json
```

Confirm it now shows `"url": "https://twinhearts.vercel.app"`. Only then rebuild the release APK in Android Studio (Build → Generate Signed Bundle/APK), and republish with:

```bash
node scripts/publish-apk.js /path/to/app-release.apk
```

One more check: your `server.js` CORS whitelist already has `https://twinhearts.vercel.app` in it, so no backend change needed there — once the sync/rebuild picks up the right domain, API calls should work too.

If you run `cap sync` and the assets file still shows the old URL, paste me the output and I'll dig into why the sync isn't picking up your config change.

Give me the all steps to rebuild my apk and all up to last step

Check scripts and keystore presence

No keystore in the repo (good — that means you're signing separately, probably via Android Studio's saved keystore on your machine). Here's the full rebuild process, start to finish:

1. Confirm the domain is correct in source

bash

```
cd us-app
cat capacitor.config.json
```

Confirm `server.url` is `https://twinhearts.vercel.app` (your current live domain). Fix it now if it isn't — this is the file that everything downstream reads from.

2. Bump the version

Open `android/app/build.gradle`, and increase both values so the Play Store/your users' devices recognize it as an update:

gradle

```
versionCode 3        // was 2 — must increase
versionName "1.2"     // was 1.1
```

3. Sync Capacitor (the step that was skipped last time)

bash

```
npx cap sync android
```

This copies `capacitor.config.json`, web assets, and any plugin changes into the Android project. Then **verify it actually worked:**

bash

```
cat android/app/src/main/assets/capacitor.config.json
```

Confirm it shows the current `twinhearts.vercel.app` URL. Do not proceed until this is correct.

4. Open the Android project

```bash
npx cap open android
```

This launches Android Studio pointed at the `android/` folder. Let Gradle finish syncing (bottom status bar) before continuing.

5. Generate a signed release APK

In Android Studio:

- **Build → Generate Signed Bundle / APK**
- Choose **APK** (not Bundle, since you're distributing the APK directly)
- Select your existing keystore file (same one used for prior releases — reusing a different keystore will make Android treat it as a different app and block updates over the old install) and enter its password
- Choose **release** build variant
- Finish the build

The output APK lands in:

```
android/app/release/app-release.apk
```

6. Publish it into the web app

```bash
node scripts/publish-apk.js android/app/release/app-release.apk
```

This copies the APK to `public/downloads/twin-hearts.apk` and rewrites `app-meta.json` with the real version/size/date — so your landing page's download button and version display update automatically.

7. Commit and deploy

```bash
git add .
git commit -m "Fix APK domain sync, bump to v1.2"
git push
```

Vercel auto-deploys on push (per your setup). Confirm the deployment finishes and check

`twinhearts.vercel.app/downloads/twin-hearts.apk` is reachable afterward.

8. Test before wide distribution

- Uninstall the old APK from your test device completely (a fresh install, not an update, avoids any cached state issues)
- Download and install the new APK
- Open it and confirm it loads the app instead of a 404

If step 3's sync still shows the wrong URL after you've confirmed the root config is right, stop there and tell me the output — that would mean something else is overriding it (e.g., a leftover `android/app/src/main/assets/capacitor.config.json` not tracked properly, or a build variant reading a different file).

You're through the hardest part already — sync worked, config is confirmed correct (`twinhearts.vercel.app`), version bumped to 1.2/build 3, and you're at the "Generate Signed App Bundle or APK" dialog with an existing signed APK visible in the analyzer (`app-release.apk`, v1.2, code 3). Here's the rest:

1. Finish the signing dialog (where you are now)

- **APK** is already selected — correct, keep it.
- Click **Next**.
- On the next screen, select **Choose existing...** and pick the **same keystore file** you used for your last release (the original `.jks` / `.keystore` — not a new one). Enter the keystore password, key alias, and key password.

⚠️ This is the one step where a mistake is unrecoverable: if you use a different keystore than your original release, Android will treat it as a completely different app, and anyone who already has the old APK installed won't be able to update — they'd have to uninstall first. If you're not 100% sure which keystore file you used originally, check `android/app/build.gradle` or your project notes before continuing, or check with me and I can help you look.

2. Select build variant and finish

- Choose **release** as the build variant.
- Check **V1** and **V2** signature versions (both, for compatibility) if asked.
- Click **Finish**. Android Studio will build and show a "APK(s) generated successfully" notification when done — click **locate** in that popup to jump straight to the file.

The output will be at:

```
android\app\release\app-release.apk
```

3. Publish it into the web app

Back in PowerShell:

```powershell
cd C:\Users\vamsi\Downloads\US
node scripts/publish-apk.js android\app\release\app-release.apk
```

This copies it to `public/downloads/twin-hearts.apk` and updates `app-meta.json` (version/size/date) automatically.

4. Commit and push

```powershell
git add .
git commit -m "Fix APK domain sync, bump to v1.2"
git push
```

Vercel will auto-deploy since it's connected to your `main` branch.

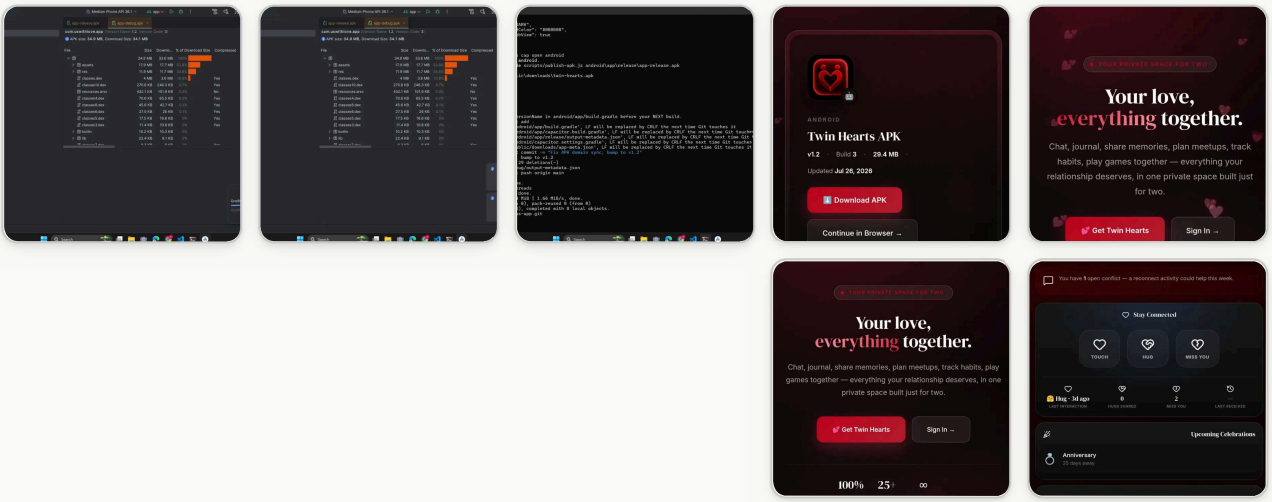
5. Verify the live download

Once the Vercel deployment shows "Ready", visit `twinhearts.vercel.app` and download the APK from there (not the old file on your PC) to make sure the deployed one is the new build.

6. Test on device

- Uninstall the old app from your phone completely first.
- Install the fresh APK.
- Open it — it should load the app instead of a 404.

Let me know once you've got it installed and I'll help if anything's still off.



Hey preety claude this is now working and thanks for helping me this understandable way.

But i have another problem in the way app is working darling can you help me.

Yay, so glad it's working now — that build/publish/deploy chain came together nicely, and the dashboard looks great too 🎉

I'll just keep it as Claude though — happy to help either way! What's going on with the app? Go ahead and describe the issue (or drop a screenshot) and I'll dig in.

You can see the flow of my app.

While user open the app in broswer then the user can see this landing page then they can use the web app or if they want they want they can download the app from this steps that's absuletly good. but after download the apk then they again comming the landing page but that is not the good flow. insted of comming the landing page again, after download the apk i want only the sign in and sign up card only like a real app flow then open the app like now.

https://github.com/vamsitulugu/us-app.git

Ran 6 commands, viewed 4 files

Now I understand the full picture. Good news: the sign-in/sign-up card you want already